Here is the implementation for **Stage 5: Swagger UI documentation with bearer auth**. This configures the OpenAPI specification so that a global lock icon appears, allowing you to authenticate once and test all protected endpoints directly from the browser.

Node.js (Express & JavaScript)

To serve Swagger UI in Express, you will need to add the `swagger-ui-express` package (`npm install swagger-ui-express`).

javascript

```javascript
import express from 'express';
import swaggerUi from 'swagger-ui-express';
import { createClient } from '@supabase/supabase-js';

const app = express();
app.use(express.json());

const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_ANON_KEY);

// ========================================
// 1. OPENAPI SPECIFICATION (WITH JWT BEARER AUTH)
// ========================================
const openapiSpecification = {
  openapi: "3.0.0",
  info: {
    title: "Supabase Auth API",
    version: "1.0.0",
    description: "API with complete signup, login, logout, and protected routes using Supa
  },
  components: {
    securitySchemes: {
      // Define the padlock configuration
      BearerAuth: {
        type: "http",
        scheme: "bearer",
        bearerFormat: "JWT",
        description: "Enter your Supabase access_token (JWT) here."
      }
    }
  },
  paths: {
    "/auth/signup": {
```

```
    post: {
      summary: "Register a new user",
      requestBody: {
        required: true,
        content: { "application/json": { schema: { type: "object", properties: { email:
      },
      responses: { 201: { description: "Created" }, 400: { description: "Bad Request" }
    }
  },
  "/auth/login": {
    post: {
      summary: "Log in an existing user",
      requestBody: {
        required: true,
        content: { "application/json": { schema: { type: "object", properties: { email:
      },
      responses: { 200: { description: "OK" }, 401: { description: "Unauthorized" } }
    }
  },
  "/public/info": {
    get: {
      summary: "Get public information",
      responses: { 200: { description: "OK" } }
    }
  },
  "/auth/logout": {
    post: {
      summary: "Log out the current user",
      security: [{ BearerAuth: [] }], // Lock icon applied here
      responses: { 204: { description: "No Content" }, 401: { description: "Unauthorized
    }
  },
  "/protected/profile": {
    get: {
      summary: "Get user profile details",
      security: [{ BearerAuth: [] }], // Lock icon applied here
      responses: { 200: { description: "OK" }, 401: { description: "Unauthorized" } }
    }
  },
  "/protected/dashboard": {
    get: {
      summary: "Get user dashboard data",
      security: [{ BearerAuth: [] }], // Lock icon applied here
      responses: { 200: { description: "OK" }, 401: { description: "Unauthorized" } }
```

```
      }
    }
  }
};

// Serve Swagger UI at /docs
app.use('/docs', swaggerUi.serve, swaggerUi.setup(openapiSpecification));

// ========================================
// 2. EXISTING MIDDLEWARE & ROUTES FROM STAGE 4
// ========================================
async function requireAuth(req, res, next) { /* ... same as Stage 4 ... */ }
app.post('/auth/signup', async (req, res) => { /* ... */ });
app.post('/auth/login', async (req, res) => { /* ... */ });
app.get('/public/info', (req, res) => { /* ... */ });
app.post('/auth/logout', requireAuth, async (req, res) => { /* ... */ });
app.get('/protected/profile', requireAuth, (req, res) => { /* ... */ });
app.get('/protected/dashboard', requireAuth, (req, res) => { /* ... */ });

app.listen(3000, () => console.log('Server running on http://localhost:3000 (Docs at /docs
```

Use code with caution.

Python (FastAPI)

FastAPI autogenerates documentation out of the box. To add the authorization locks to your `/docs` interface, swap out the raw header extraction for FastAPI's native `HTTPBearer` security dependency.

python

```
import os
from fastapi import FastAPI, HTTPException, status, Depends, Request
from fastapi.responses import JSONResponse
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from supabase import create_client, Client

app = FastAPI(
    title="Supabase Auth API",
```

```
    description="API with complete signup, login, logout, and protected routes using Supab
)
supabase: Client = create_client(os.environ.get("SUPABASE_URL"), os.environ.get("SUPABASE_

# 1. Initialize FastAPI's native Bearer scheme
security_scheme = HTTPBearer()

# ========================================
# 2. UPDATED DEPENDENCY GUARD
# ========================================
def get_current_user(credentials: HTTPAuthorizationCredentials = Depends(security_scheme))
    # FastAPI automatically handles header parsing and enforces structural check.
    # If the 'Authorization: Bearer <token>' header is missing, FastAPI automatically
    # yields a 403. To match your custom 401 requirement precisely, we intercept errors or
    token = credentials.credentials

    try:
        response = supabase.auth.get_user(token)
        user = response.user
        if not user:
            raise ValueError()

        return {"user": user, "token": token}
    except Exception:
        raise HTTPException(status_code=401, detail="Invalid or expired token")

# Override to ensure exact API response JSON compliance
@app.exception_handler(HTTPException)
async def custom_http_exception_handler(request: Request, exc: HTTPException):
    return JSONResponse(status_code=exc.status_code, content={"error": exc.detail})

# ========================================
# 3. PUBLIC & AUTH ENDPOINTS
# ========================================
@app.post("/auth/signup")
def signup(): pass

@app.post("/auth/login")
def login(): pass

@app.get("/public/info")
def public_info(): pass

# ========================================
```

```python
# 4. PROTECTED ENDPOINTS (Locks automatically applied via Depends)
# ========================================
@app.post("/auth/logout", status_code=status.HTTP_204_NO_CONTENT)
def logout(current_auth: dict = Depends(get_current_user)):
    try:
        supabase.auth.sign_out(current_auth["token"])
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))
    return None

@app.get("/protected/profile")
def get_protected_profile(current_auth: dict = Depends(get_current_user)):
    user = current_auth["user"]
    return {
        "id": user.id,
        "email": user.email,
        "created_at": user.created_at
    }

@app.get("/protected/dashboard")
def get_protected_dashboard(current_auth: dict = Depends(get_current_user)):
    user = current_auth["user"]
    return {
        "message": f"Welcome to your dashboard, {user.email}!",
        "stats": {"premium": True}
    }
```

Use code with caution.

Now you can open `http://localhost:3000/docs` (or port `8000` for Python), click the global **Authorize** button at the top right, paste your JWT, and click "Try it out" across your locked endpoints!

What should we explore next to finalize your API setup?

- Add **refresh token handling** to swap expired access tokens without re-authenticating
- Implement automated **PostgreSQL Row Level Security (RLS)** in Supabase to guard database entries
- Write a **Postman Collection** or integration test suite for this spec